

Trust the Query, Not the Answer

Auditable Text-to-SPARQL over the GLEIF Register

Jannic Alexander Cutura | European Central Bank | DSTI School of Engineering

github.com/JannicCutura/finnlp-2026

1. The problem

In anti-money-laundering screening, credit-risk and supervision an answer is useful only if it traces back to a source record. Ask a chat assistant which entities Siemens consolidates and it returns a dozen plausible names, none carrying the identifier you would need to check it. Ask the register and you get exactly **452** entities reporting Siemens Aktiengesellschaft as their direct consolidation parent, each traceable to its record by LEI.

So do not trust the answer. Trust the query, and read it.

2. The register

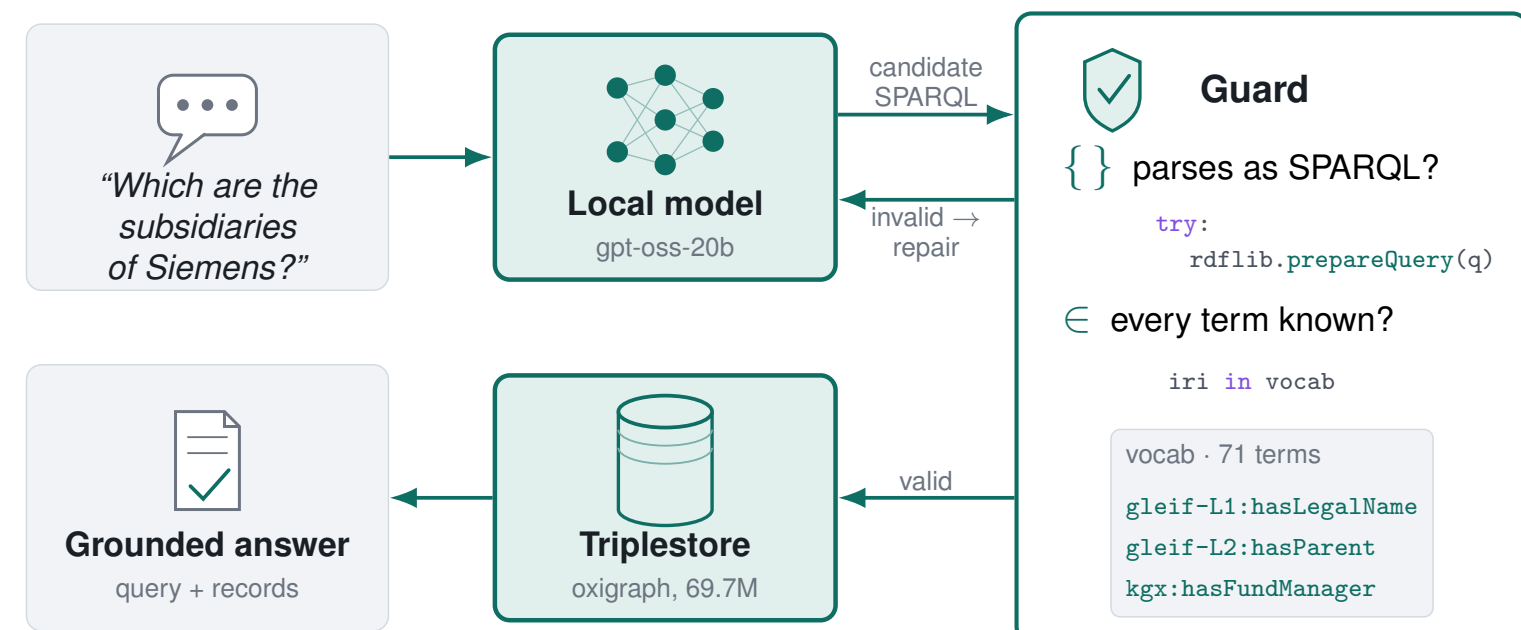
GLEIF publishes the Legal Entity Identifier register as CC0 open data with an official RDF ontology — a regulatory source, not an encyclopaedic or synthetic graph. We pin one 8-hourly publish and query it locally.

Level 1 entity records	3,382,301
of which ISSUED	1,940,941
Level 2 relationships	481,557
Triples in the store	69.7 M
Populated predicates	27
Guard vocabulary	71 terms
Snapshot	20260723-1600

The guard vocabulary is everything a query may name: all 27 populated predicates plus 44 classes and code values, each count-backed from the graph.

3. The system

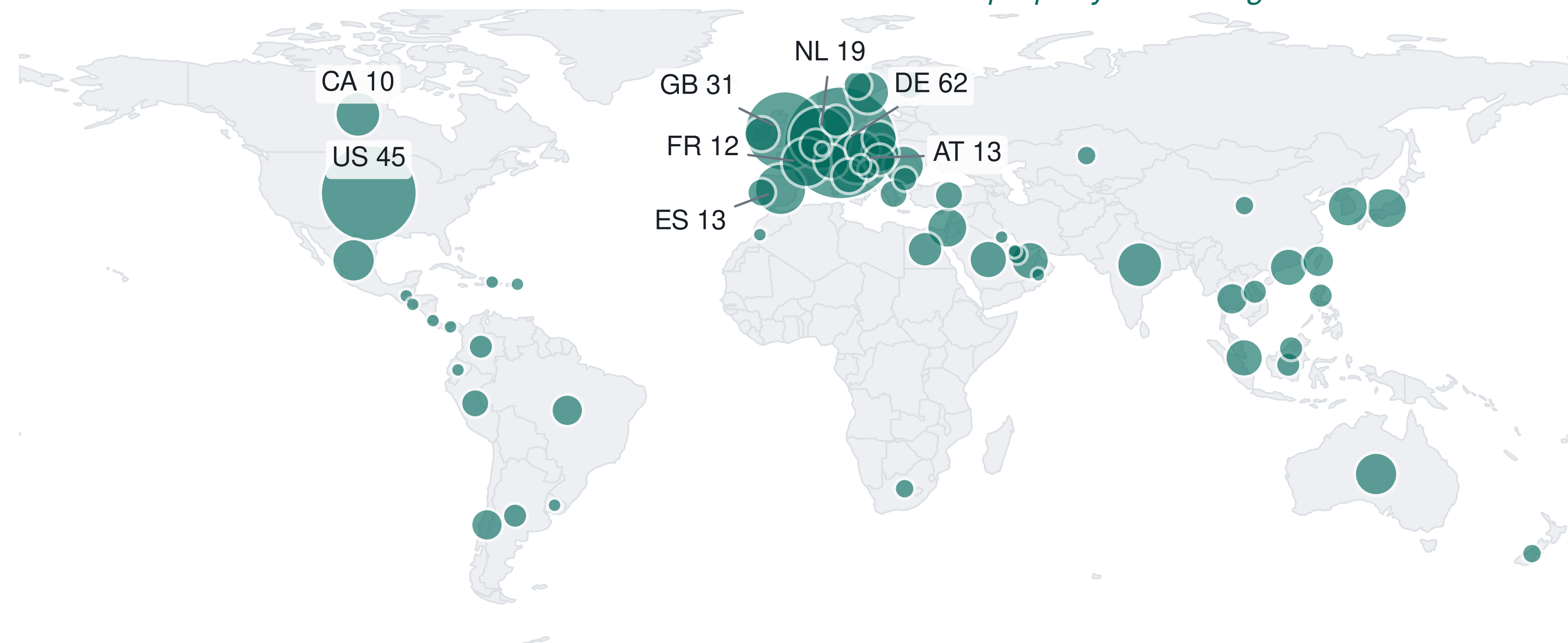
A local open-weight model (gpt-oss-20b, served through LM Studio at temperature 0, prompting only, no training) turns the question into candidate SPARQL. A guard checks the query *before* it runs; a bounded repair loop of at most two retries feeds its diagnostic back to the model. The query, its rows and the LEIs behind them return with every answer.



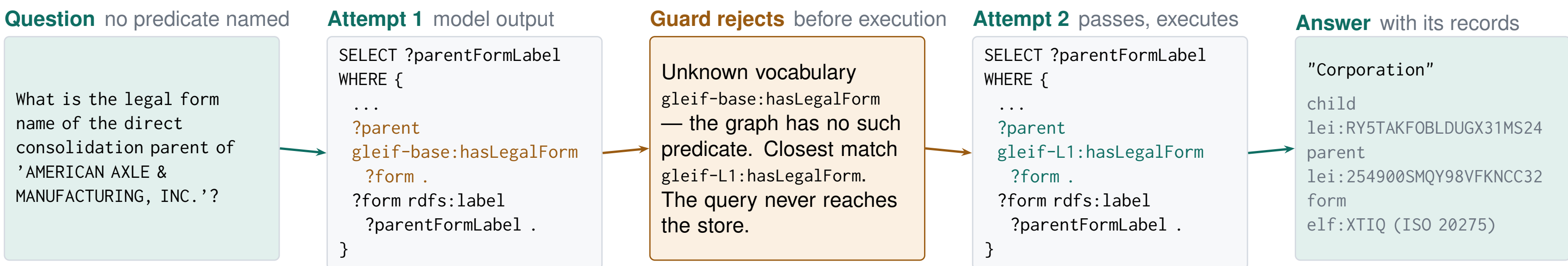
One shot, not agentic search: the guard, not iterative graph exploration, keeps the output valid. The loop never retries on an empty result — for an unanswerable question, empty is right.

4. Cost

Queries return well under a second, except full-population aggregates (14–28s). Generation, not the store, dominates latency (15–40s). The whole system runs on one machine and never calls out, so the data stays on premises.



452 direct subsidiaries of Siemens Aktiengesellschaft, by country of registration — circle area proportional to count.



The guard rejects `gleif-base:hasLegalForm`: no such predicate in the graph. The diagnostic and its closest match go back to the model, which repairs the line and the query executes.

5. One question, end to end

A held-out item (h55), exactly as the system logged it above. Like most repairs, the fix is a one-token vocabulary swap; the guard's diagnostic already names the correct replacement, which is why a single retry is usually enough.

6. What the guard does not do

The guard is an existence check, not a semantic one. It guarantees that the query parses and that every term it names is in the graph, which is what stops a hallucinated predicate from ever reaching the store. It does **not** guarantee that the query means what the question asked. A stricter check could compare predicate domains and ranges against the ontology, catching some of what slips through here — at the cost of also rejecting correct queries the schema does not anticipate.

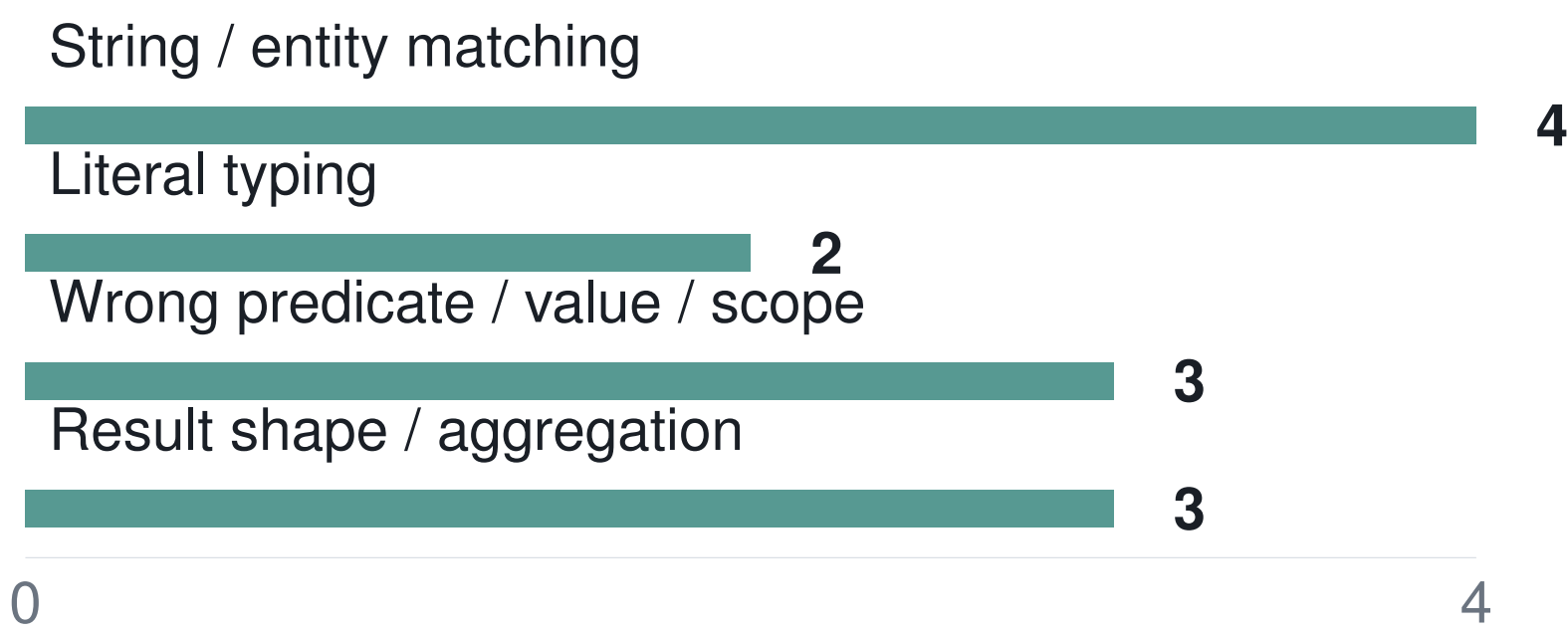
Checks: the query parses as SPARQL; every predicate, class and code value it names is in the 71-term vocabulary.

Does not check: whether that predicate is the right one, whether a filter or aggregation matches the question, or whether the matched entity is the one intended.

That boundary is the design and not a gap in it: a wrong answer arrives as a readable query returning wrong rows, never as fluent prose.

7. Where the errors are not

All twelve held-out errors are queries that ran and returned the wrong rows. None is invalid, none names a predicate that does not exist, and the model declines every out-of-scope and absent-entity question it is given. Development errors cluster the same way, which is why later schema revisions added vocabulary coverage rather than tried to make the guard itself smarter.



All twelve errors, by type; none is a vocabulary or schema failure.

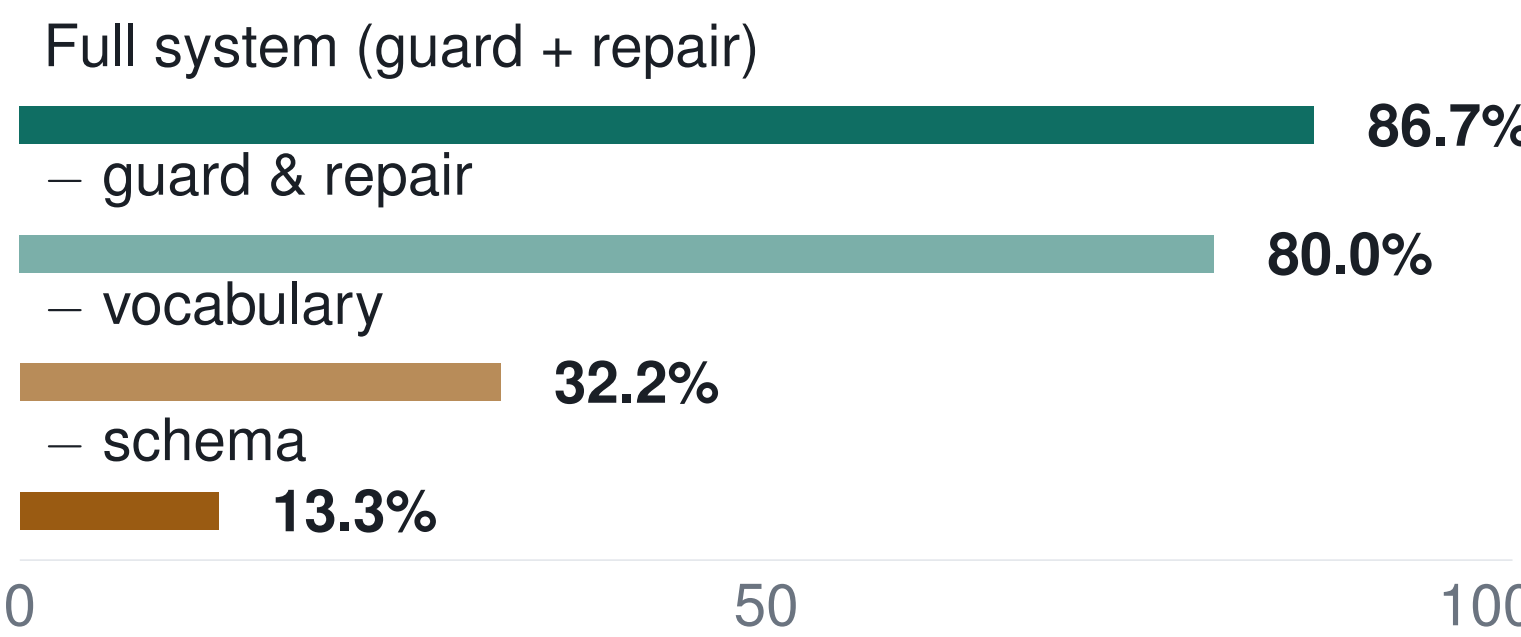
Every one of the twelve is visible in the query itself, which is the property the setting demanded.

8. Benchmark

144 questions over six tiers, from a specification fixed *before* instantiation around a KYC analyst, a supervisor and a fund-operations user, in business language naming no predicate or IRI. Every reference query runs against the pinned snapshot, so its result cannot drift from the graph. The 54-question development set drove three schema revisions; the 90-question held-out set came afterwards, against a frozen prompt.

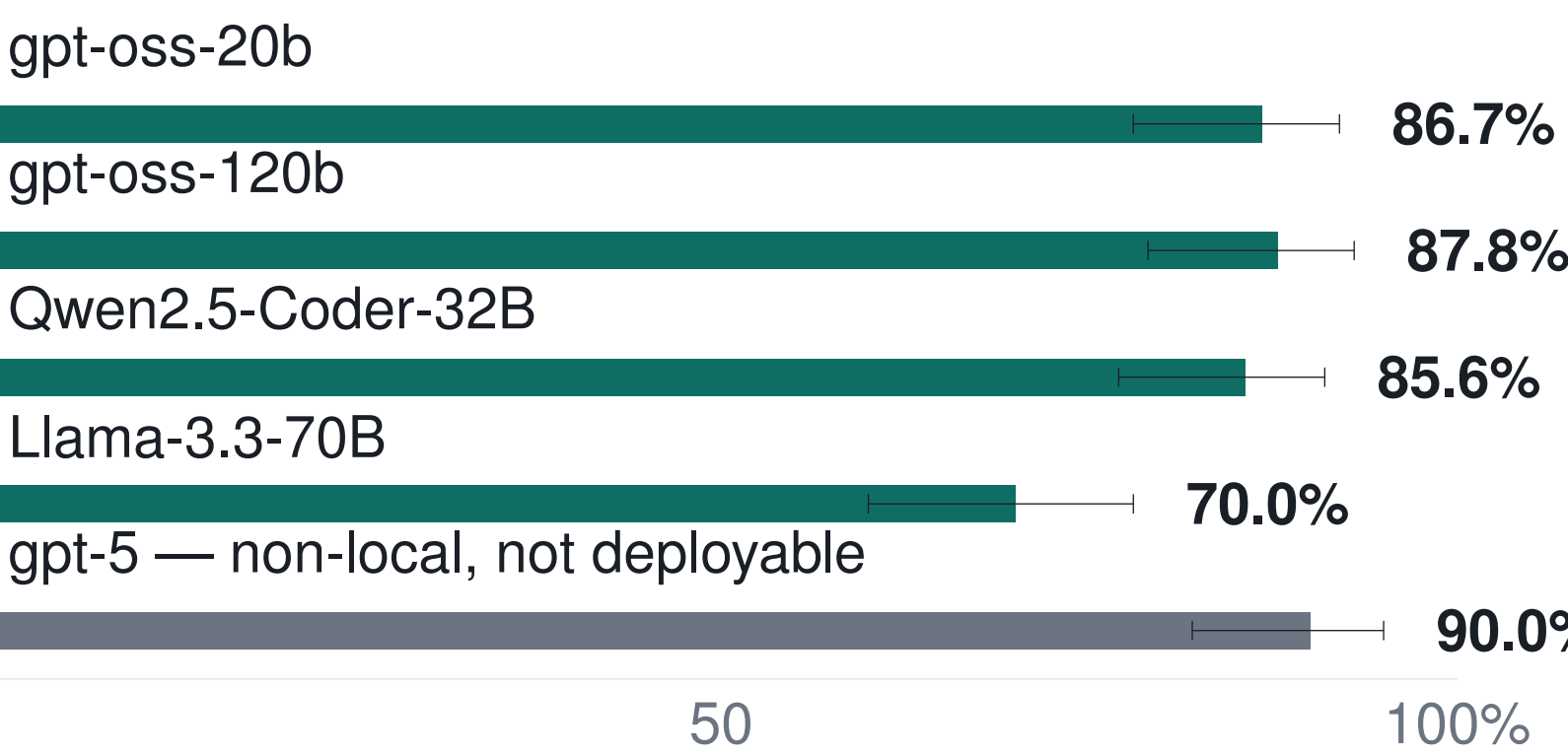
	20B		120B
Tier	Dev	Held	Held
a lookup	10/10	10/15	12/15
b single-hop	10/10	14/14	14/14
c multi-hop	8/8	13/15	14/15
d attribute filter	7/7	14/15	14/15
e aggregate	6/7	13/15	11/15
f unanswerable	12/12	14/16	14/16
Exec. accuracy	98.1%	86.7%	87.8%
95% CI	90–100	78–92	79–93
QALD macro-F1	0.981	0.897	0.893
Validity rate	100%	100%	98.9%
First-try / repaired	48/6	82/8	80/9

9. What each part buys



Held-out accuracy (20B): stripping vocabulary drops it to 32.2%, stripping schema to 13.3%. Guard and repair add the last 6.7 points, to 100% validity.

10. Models



95% CI whiskers: size is not the bottleneck and open versus closed is not either; training focus is. The code-tuned models sit inside one another's intervals, 16 points above the general instruction model.

11. Limits, and what is next

144 questions on one graph with a thin schema, one snapshot, one run per model, and no user study yet. The construction, though, is not specific to corporate structure: it is a model writing queries against any source described by an ontology. Real data sits across relational, graph and document stores, each with its own query language — extending the same generate–guard–repair loop across them is the direction we intend to pursue. The nearer-term test is whether the same guard holds up on a schema an order of magnitude larger, where a single typo has far more plausible near-misses to repair towards.

86.7% of 90 held-out questions answered correctly, every one with a valid query behind it — and every error visible in that query.

Code, prompts and benchmark
github.com/JannicCutura/finnlp-2026

The views expressed in this poster are those of the author and do not necessarily reflect those of the European Central Bank or the Eurosystem.